

FULL STACK PYTHON DEVELOPMENT WITH AI

Course Intro:

In this introduction module, you will learn about the complete course structure, learning objectives, and the skills you will gain throughout the training. The course is designed to provide both theoretical knowledge and practical experience through hands-on exercises, real-world examples, and industry-oriented projects.

By the end of this course, you will be able to understand core concepts, build practical applications, solve real-world problems, and develop job-ready skills. You will also learn industry best practices, project development techniques, and tools commonly used by professionals.

Benefits of This Course

- Learn concepts from beginner to advanced level.
- Gain hands-on experience through practical examples and projects.
- Develop problem-solving and logical thinking skills.
- Build real-world applications and portfolio projects.
- Understand industry standards and best practices.
- Improve coding, debugging, and project development skills.
- Prepare for technical interviews and career opportunities.
- Receive comprehensive knowledge that can be applied in professional environments.

This course is suitable for students, job seekers, working professionals, and anyone interested in developing practical skills and advancing their career.

Python Introduction

• Introduction to Programming

- What is Programming?
- Importance of Programming in Modern Technology
- Problem Solving Through Programming
- Programming Languages Overview
- Applications of Programming

• Introduction to Coding

- What is Coding?
- Coding vs Programming
- Role of a Programmer
- Writing and Executing Code
- Best Practices for Beginners

• Introduction to Python

- History of Python
- Features of Python
- Python as an Interpreted Language
- Applications of Python in Industry
- Python Ecosystem Overview

• Python Libraries

- What are Libraries?
- Importance of Libraries in Development
- Popular Python Libraries
- NumPy, Pandas, Matplotlib Overview
- Installing Libraries Using PIP

• Python Modules

- What is a Module?
- Built-in Modules in Python
- Importing Modules
- Creating User-Defined Modules
- Benefits of Modular Programming

• Python Web Frameworks

- Introduction to Web Frameworks
- Django Framework Overview
- Flask Framework Overview
- Choosing the Right Framework

- **What Can Python Do?**

- Web Development
- Data Science and Analytics
- Artificial Intelligence and Machine Learning
- Automation and Scripting
- Desktop Application Development
- Game Development
- Cybersecurity and Networking

- **Why Python?**

- Easy to Learn and Read
- Large Community Support
- Cross-Platform Compatibility
- Rich Library Ecosystem
- High Demand in Industry
- Rapid Application Development

- **Python Syntax Compared to Other Languages**

- Python vs C
- Python vs Java
- Python vs JavaScript
- Simplicity of Python Syntax
- Code Readability and Maintainability

- **Python Installation and Setup**

- Downloading Python
- Installing Python on Windows
- Installing Python on Linux
- Installing Python on macOS
- Setting Environment Variables
- Verifying Installation
- Using IDLE and VS Code

- **Development Environment Setup**

- Introduction to IDEs and Code Editors
- Visual Studio Code Setup
- Running Python Programs
- Managing Python Projects

Software Requirements IDE installations

Fundamentals of Python

• The print() Statement

- Introduction to print() Function
- Displaying Text and Variables
- Printing Multiple Values
- Formatting Output for Readability

• Comments in Python

- Purpose of Comments
- Single-Line Comments (#)
- Multi-Line Comments
- Best Practices for Code Documentation

• Python Keywords

- Introduction to Reserved Keywords
- Common Python Keywords
- Using the keyword Module
- Restrictions on Keyword Usage

• Python Variables

- What are Variables?
- Variable Declaration and Assignment
- Naming Rules and Conventions
- Multiple Variable Assignment
- Dynamic Typing in Python

• Python Data Types

- Numeric Data Types (int, float, complex)
- Boolean Data Type
- String Data Type
- None Data Type
- Checking Data Types Using type()

• Python Data Structures

- Introduction to Data Structures
- Lists
- Tuples
- Sets
- Dictionaries
- Choosing the Right Data Structure

• Python Type Conversions

- Implicit Type Conversion
- Explicit Type Conversion
- Converting Between int, float, and str
- Converting Collections (list, tuple, set)
- Handling Conversion Errors

• Simple Input and Output

- Using input() Function
- Reading User Input
- Type Casting User Input
- Interactive Programs
- Displaying Formatted Output

Python Operators

• Introduction to Operators

- What are Operators?
- Importance of Operators in Programming
- Operands and Expressions
- Real-Time Applications of Operators

• Arithmetic Operators

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Floor Division (//)
- Modulus (%)
- Exponentiation (**)
- Arithmetic Expressions and Calculations

• Assignment Operators

- Assignment Operator (=)
- Add and Assign (+=)
- Subtract and Assign (-=)
- Multiply and Assign (*=)
- Divide and Assign (/=)
- Floor Divide and Assign (//=)
- Modulus and Assign (%=)
- Exponent and Assign (**=)

• Comparison Operators

- Equal To (==)

- Not Equal To (!=)
- Greater Than (>)
- Less Than (<)
- Greater Than or Equal To (>=)
- Less Than or Equal To (<=)
- Using Comparison Operators in Decision Making

• Logical Operators

- and Operator
- or Operator
- not Operator
- Combining Multiple Conditions
- Truth Tables and Boolean Logic

• Identity Operators

- is Operator
- is not Operator
- Object Identity vs Equality
- Practical Examples of Identity Checking

• Membership Operators

- in Operator
- not in Operator
- Membership Testing in Strings
- Membership Testing in Lists, Tuples, and Dictionaries
- Real-World Applications

Python Conditional Statement

• Introduction to Indentation

- What is Indentation?
- Importance of Indentation in Python
- Indentation Rules and Best Practices
- Common Indentation Errors
- Writing Clean and Readable Code

• Introduction to Conditional Statements

- What are Conditional Statements?
- Decision Making in Python
- Boolean Expressions and Conditions
- Comparison and Logical Operators
- Real-Time Applications of Conditional Logic

- **The if Statement**

- Introduction to if Statement
- Syntax and Structure
- Evaluating Conditions
- Using Multiple Conditions
- Simple Decision-Making Programs

- **Related Statements of if**

- if-else Statement
- if-elif-else Statement
- Multiple elif Conditions
- Decision Trees and Branching Logic

- **Examples with if and Related Statements**

- Positive or Negative Number Check
- Even or Odd Number Program
- Age Eligibility Verification
- Grade Classification System
- Simple Calculator Logic

- **Else Statement**

- Introduction to else
- Alternative Execution Paths
- Using else with if Statements
- Practical Examples

- **Nested If Statements**

- Introduction to Nested if
- Multiple Levels of Decision Making
- Combining Conditions
- Real-Time Examples of Nested Logic

- **Short Hand If**

- One-Line if Statements
- Syntax and Usage
- Writing Compact Conditional Logic
- Best Practices for Readability

- **Short Hand If-Else (Ternary Operator)**

- Introduction to Ternary Operators
- Syntax and Structure
- Replacing Traditional if-else Statements
- Practical Examples

Python Iterative Statements

• Introduction to Indentation

- What is Indentation?
- Importance of Indentation in Python
- Indentation Rules and Standards
- Common Indentation Errors
- Writing Readable and Structured Code

• Introduction to Loops

- What are Loops?
- Need for Iteration in Programming
- Types of Loops in Python
- Loop Control Flow
- Real-Time Applications of Loops

• For Loop

- Introduction to for Statement
- Syntax and Structure
- Iterating Through Strings
- Iterating Through Lists, Tuples, and Dictionaries
- Using range() Function
- Looping with Start, Stop, and Step Values

• Examples with For Loop

- Printing Numbers Using for Loop
- Multiplication Tables
- Sum of Numbers
- Factorial Calculation
- Pattern Printing Programs

• While Loop

- Introduction to while Statement
- Syntax and Structure
- Condition-Based Iteration
- Infinite Loops and Prevention
- Using else with while Loop
- Examples with While Loop

• Nested For Loops

- Introduction to Nested Loops
- Syntax of Nested for Loops

• Nested While Loops

- Syntax of Nested while Loops
- Controlling Multiple Conditions
- Practical Examples

Python Jumping Statements

• Introduction to Python Indentation

- What is Indentation?
- Importance of Indentation in Python
- Indentation Rules and Best Practices
- Code Readability and Maintainability

• Control Flow in Python

- Understanding Program Execution Flow
- Conditional Statements and Loops
- Role of Jump Statements
- Managing Program Logic Efficiently

• Break Statement

- Introduction to break
- Working of break in Loops
- Using break with for Loops
- Using break with while Loops
- Nested Loop Examples
- Real-Time Applications of break

• Continue Statement

- Introduction to continue
- Skipping Iterations in Loops
- Using continue with for Loops
- Using continue with while Loops
- Practical Examples and Use Cases

• Pass Statement

- Introduction to pass
- Placeholder Statements

Python Strings

• String Introduction

- What is a String?
- Characteristics of Strings
- Creating Strings in Python
- Single Quotes, Double Quotes, and Triple Quotes
- Real-Time Applications of Strings

• String Object Basics

- String Immutability
- Accessing Characters Using Indexing
- Positive and Negative Indexing
- String Slicing
- Traversing Strings Using Loops
- Membership Operators (in, not in)

• String Methods

- upper() and lower()
- capitalize() and title()
- strip(), lstrip(), and rstrip()
- replace()
- find() and index()
- count()
- startswith() and endswith()
- isalpha(), isdigit(), isalnum()
- islower() and isupper()

• String Operations

- String Concatenation
- String Repetition
- String Comparison
- Escape Characters
- Raw Strings

• Splitting and Joining Strings

- split() Method
- rsplit() Method
- splitlines() Method
- join() Method
- Converting Strings into Lists
- Combining Collections into Strings

• String Formatting Functions

- `format()` Method
- f-Strings (Formatted String Literals)

Python List

• Introduction to Lists

- What is a List?
- Characteristics of Lists
- Creating Lists
- Accessing List Elements
- Positive and Negative Indexing
- Real-Time Applications of Lists

• List Basics

- Adding Elements to Lists
- Updating List Elements
- Deleting List Elements
- List Slicing
- Traversing Lists using Loops
- Membership Operators (`in`, `not in`)

• List Methods

- `append()`
- `extend()`
- `insert()`
- `remove()`
- `pop()`
- `clear()`
- `index()`
- `count()`
- `sort()`
- `reverse()`
- `copy()`

• Nested Lists

- Introduction to Nested Lists
- Creating Multi-Dimensional Lists
- Accessing Nested Elements

• List Comprehensions

- Introduction to List Comprehensions
- Syntax and Structure
- Conditional List Comprehensions
- Nested List Comprehensions
- Performance Benefits

- Real-Time Examples
- **Advanced List Operations**
 - List Concatenation
 - List Repetition
 - Sorting Complex Lists
 - Filtering List Data

Python Tuples

- **Introduction to Tuples**
 - What is a Tuple?
 - Characteristics of Tuples
 - Immutable Nature of Tuples
 - Creating Tuples
 - Single-Element Tuples
 - Real-Time Applications of Tuples
- **Working with Tuple Elements**
 - Accessing Tuple Elements
 - Positive and Negative Indexing
 - Tuple Slicing
 - Iterating Through Tuples
 - Nested Tuples
- **Tuple Built-in Functions**
 - len()
 - max()
 - min()
 - sum()
 - sorted()
 - tuple()
 - any()
 - all()
- **Tuple Methods**
 - count()
 - index()
 - Searching Elements in Tuples
 - Counting Occurrences of Values
- **Tuple Operations**
 - Concatenation of Tuples

- Repetition Operator (*)
 - Membership Operators (in, not in)
 - Comparison Operations
 - Tuple Packing and Unpacking
- **Advanced Tuple Concepts**
 - Tuple Packing
 - Tuple Unpacking
 - Multiple Variable Assignment
 - Using Tuples as Function Return Values
 - Nested Tuple Operations

Python Sets

- **Introduction to Sets**
 - What is a Set?
 - Features of Sets
 - Characteristics of Unordered Collections
 - Unique Elements in Sets
 - Creating Sets in Python
 - Real-Time Applications of Sets
- **Working with Set Elements**
 - Accessing Set Elements
 - Adding Elements using add()
 - Adding Multiple Elements using update()
 - Removing Elements using remove()
 - Using discard() and pop()
 - Clearing and Deleting Sets
- **Set Built-in Functions and Methods**
 - len()
 - max()
 - min()
 - sum()
 - sorted()
 - copy()
 - clear()
 - isdisjoint()
- **Set Operations**
 - Union Operation
 - Intersection Operation
 - Difference Operation

- Symmetric Difference
- Subset and Superset Operations
- Membership Testing
- Comparing Sets

- **Advanced Set Methods**

- union()
- intersection()
- difference()
- symmetric_difference()
- issubset()
- issuperset()
- intersection_update()
- difference_update()

Python FrozenSet

- **Introduction to FrozenSet**

- What is a FrozenSet?
- Features of FrozenSet
- Difference Between Set and FrozenSet
- When to Use FrozenSet
- Real-World Applications of FrozenSet

- **Creating FrozenSets**

- Using the frozenset() Function
- Creating FrozenSets from Lists
- Creating FrozenSets from Tuples
- Creating FrozenSets from Sets

Python Dictionary

- **Introduction to Dictionaries**

- What is a Dictionary?
- Characteristics of Dictionaries
- Key-Value Pair Concept
- Creating Dictionaries
- Real-Time Examples of Dictionaries

- **Working with Dictionary Elements**

- Accessing Dictionary Values
- Adding New Key-Value Pairs
- Updating Existing Values
- Deleting Dictionary Items
- Nested Dictionaries

- **Dictionary Built-in Functions and Methods**

- len()
- type()
- keys()
- values()
- items()
- get()
- update()
- pop()
- clear()
- copy()

Python Functions

• Introduction to Functions

- What is a Function?
- Benefits of Using Functions
- Code Reusability and Modularity
- Real-World Applications of Functions

• Defining Functions

- Creating Functions Using def Keyword
- Function Syntax and Structure
- Naming Conventions for Functions
- Writing Reusable Functions

• Calling Functions

- Invoking Functions
- Function Execution Flow
- Calling Functions Multiple Times
- Functions within Functions

• Return Statement

- Understanding return Keyword
- Returning Single Values
- Returning Multiple Values
- Returning Objects and Collections
- Best Practices for Using return

• Difference Between return and print

- Purpose of return Statement
- Purpose of print() Function
- Output vs Returned Value
- Practical Examples and Comparisons

• Function Arguments and Parameters

- Understanding Parameters
- Understanding Arguments
- Positional Arguments
- Passing Values to Functions
- Function Parameter Scope

• Keyword Arguments

- Named Parameter Passing
- Default Arguments

- Mixing Positional and Keyword Arguments
- Practical Examples

- **Arbitrary Arguments**

- *args (Variable-Length Arguments)
- **kwargs (Keyword Variable-Length Arguments)
- Handling Dynamic Input Values
- Real-Time Use Cases

- **User-Defined Functions**

- Creating Custom Functions
- Business Logic Implementation
- Reusable Utility Functions
- Function Libraries and Organization

- **Nested Functions**

- Functions Inside Functions
- Variable Scope in Nested Functions
- Closures Introduction
- Practical Applications of Nested Functions

- **Variable Scope in Functions**

- Local Variables
- Global Variables

- **Mini Projects Using Functions**

- ATM Simulation Program

Python Advance Functions

• Introduction to Advanced Functions

- Overview of Functional Programming Concepts
- Functions as First-Class Objects
- Benefits of Functional Programming
- Real-World Applications

• Lambda Functions

- What are Lambda Functions?
- Syntax and Structure
- Anonymous Functions
- Using Lambda with Conditional Expressions
- Real-Time Examples of Lambda Functions

• Map Function

- Introduction to map()
- Applying Functions to Iterables
- Using Lambda with map()
- Working with Multiple Iterables
- Practical Data Transformation Examples

• Filter Function

- Introduction to filter()
- Filtering Data Based on Conditions
- Using Lambda with filter()
- Filtering Lists and Collections
- Real-World Data Validation Examples

• Reduce Function

- Introduction to functools.reduce()
- Aggregating Data
- Using Lambda with reduce()
- Performing Cumulative Calculations
- Business and Statistical Use Cases

• Return Statement vs Print Function

- Understanding return Statement
- Understanding print() Function
- Key Differences Between return and print

- When to Use return vs print
- Function Design Best Practices

- **Return Statement vs Yield Keyword**

- How return Works
- How yield Works
- Memory Usage Comparison
- Performance Considerations
- Use Cases for return and yield

- **Generators in Python**

- Introduction to Generators
- Creating Generator Functions
- Generator Expressions
- Lazy Evaluation Concept
- Benefits of Generators

- **Yield Keyword**

- Understanding yield
- Creating Iterators with yield
- State Preservation in Generators
- Multiple Yield Statements
- Generator Execution Flow

OOP's concept with Python

• Introduction to Object-Oriented Programming

- What is OOP?
- Procedural Programming vs Object-Oriented Programming
- Benefits of OOP
- Real-World Applications of OOP

• Classes and Objects

- Introduction to Classes
- Creating Classes
- Class Attributes and Methods
- Real-Time Examples of Classes
- Creating Objects
- Accessing Class Members

• Constructors and Special Methods

- Introduction to Constructors
- `__init__()` Method
- Parameterized Constructors
- Default Constructors
- Object Initialization Techniques

• self and super Keywords

- Understanding self Keyword
- Accessing Instance Variables and Methods
- Understanding super() Function
- Calling Parent Class Constructors
- Accessing Parent Class Methods

• Inheritance

- Introduction to Inheritance
- Code Reusability Through Inheritance
- Creating Parent and Child Classes
- Real-World Examples of Inheritance

• Types of Inheritance

- Single Inheritance
- Multiple Inheritance
- Multi-Level Inheritance

- Hierarchical Inheritance

- **Polymorphism**

- Introduction to Polymorphism
- Method Overloading (Concept in Python)
- Method Overriding
- Operator Overloading
- Runtime Polymorphism
- Real-Time Examples of Polymorphism

- **Encapsulation**

- Introduction to Encapsulation
- Public Members
- Protected Members
- Private Members

- **Data Abstraction**

- Introduction to Abstraction
- Abstract Base Classes (ABC)
- Abstract Methods
- Implementing Abstraction in Python
- Real-Time Examples of Data Abstraction

File Handling with Python

- **Introduction to File Handling**

- What is File Handling?
- Importance of File Operations
- Real-World Applications of File Handling

- **File Modes**

- Read Mode (r)
- Write Mode (w)
- Append Mode (a)
- Read and Write Mode (r+)
- Write and Read Mode (w+)
- Append and Read Mode (a+)

- **Working with Files**

- Opening Files Using open()
- Closing Files Using close()
- Checking File Existence
- Handling File Paths
- **Using the with Keyword**
 - Context Managers in Python
 - Automatic File Closing
 - Benefits of Using with Statement
 - Best Practices for File Operations
- **Reading Files**
 - read() Method
 - Reading Entire File Content
 - Reading Specific Number of Characters
 - Handling Large Files Efficiently
- **Writing Files**
 - write() Method
 - Creating New Files
 - Appending Data to Existing Files
 - Writing Structured Data
- **Reading Files Line by Line**
 - readline() Function
 - Reading Single Lines
 - Processing Text Data
- **Reading Multiple Lines**
 - readlines() Function
 - Working with Lists of Lines
 - Iterating Through File Content
- **Writing Multiple Lines**
 - writelines() Function
 - Writing Lists to Files
 - Formatting Output Data
- **File Positioning Functions**

- tell() Function
 - Understanding File Pointers
 - Tracking Current Position
 - Practical Examples
- **Seek Function**
 - seek() Function Overview
 - Moving File Pointer
 - Random File Access
 - Reading Data from Specific Positions

Exception Handling with Python

- **Using Standard Modules**
 - Introduction to Python Modules
 - Importing Built-in Modules
 - Working with math Module
 - Working with random Module
 - Working with datetime Module
 - Working with os and sys Modules
 - Module Aliasing and Import Techniques
- **Creating Custom Modules**
 - What is a User-Defined Module?
 - Creating and Saving Python Modules
 - Importing Custom Modules
 - Using Functions from Modules
 - Module Reusability and Best Practices
- **Exception Handling**
 - Introduction to Exceptions
 - Common Runtime Errors
 - Understanding Exception Types
 - Error Handling Best Practices
- **Try-Except Blocks**
 - Using try and except Statements
 - Nested Try-Except Blocks
 - Using else Block
 - Using finally Block

Python Numpy

• Introduction to NumPy

- What is NumPy?
- Importance of NumPy in Data Science
- Applications of NumPy
- Advantages over Python Lists

• NumPy Installation and Setup

- Installing NumPy using PIP
- Importing NumPy Library
- Verifying Installation
- Setting Up Development Environment

• NumPy Nddarray Object

- Introduction to ndarray
- Creating Arrays
- One-Dimensional Arrays
- Multi-Dimensional Arrays
- Array Properties and Methods

• NumPy Data Types

- Understanding NumPy Data Types
- Integer, Float, Boolean Types
- Type Conversion and Casting

• Array Attributes

- shape
- ndim
- size
- dtype

• Array Creation Techniques

- Creating Arrays with array()
- zeros() and ones()
- empty() and full()
- identity() and eye()
- Random Array Generation

• Arrays from Existing Data

- Creating Arrays from Lists
- Creating Arrays from Tuples
- Converting Existing Data Structures
- Copying and Referencing Data

- **Arrays from Numerical Ranges**
 - `arange()`

- **Indexing and Slicing**
 - Accessing Array Elements
 - Row and Column Selection
 - Array Slicing Techniques
 - Multi-Dimensional Indexing

- **Array Manipulation**
 - Reshaping Arrays
 - Flattening Arrays
 - Transpose Operations
 - Stacking Arrays

- **Mathematical Functions**
 - Trigonometric Functions
 - Exponential and Logarithmic Functions
 - Rounding Functions
 - Mathematical Constants

- **Arithmetic Operations**
 - Addition and Subtraction
 - Multiplication and Division
 - Power and Modulus Operations
 - Element-wise Calculations

- **Statistical Functions**
 - Mean, Median, and Mode Concepts
 - Standard Deviation
 - Minimum and Maximum Values

- **Sorting, Searching, and Counting**
 - Sorting Arrays

- Searching Elements
- **Copies and Views**
 - Understanding Memory Management
 - Creating Copies
 - Creating Views
 - Difference Between Copies and Views
 - Performance Considerations
- **NumPy Matrix Library**
 - Introduction to Matrices
 - Creating Matrix Objects
 - Matrix Addition and Multiplication
 - Transpose and Inverse

Python Pandas

- **Introduction to Pandas**
 - What is Pandas?
 - Features and Advantages of Pandas
 - Applications of Pandas in Data Analysis
 - Understanding Structured Data
- **Pandas Installation and Setup**
 - Installing Pandas using PIP
 - Importing Pandas Library
 - Verifying Installation
 - Working with Jupyter Notebook
- **Pandas Series**
 - Introduction to Series
 - Creating Series Objects
 - Accessing and Modifying Data
 - Series Operations and Methods
- **Pandas DataFrame**
 - Introduction to DataFrames
 - Creating DataFrames
 - Loading Data from Dictionaries and Lists

- Accessing Rows and Columns
- DataFrame Attributes and Methods
- **Basic DataFrame Functionality**
 - Viewing and Exploring Data
 - Data Inspection Techniques
 - Handling Missing Values
 - Adding and Removing Columns
 - Updating Data
- **Indexing and Selecting Data**
 - Indexing Concepts
 - Using loc[] and iloc[]
 - Filtering Data
 - Conditional Selection
 - Slicing DataFrames
- **Descriptive Statistics**
 - Summary Statistics
 - Mean, Median, and Mode
 - Min, Max, and Range
 - Data Distribution Analysis
- **Function Application**
 - Using apply() Method
 - Lambda Functions
 - Column-wise Operations
 - Custom Functions on DataFrames
- **Reindexing and Data Alignment**
 - Understanding Reindexing
 - Changing Row and Column Labels
 - Handling Missing Index Values
 - Data Alignment Concepts
- **Categorical Data**
 - Understanding Categorical Variables
 - Creating Categorical Data
 - Managing Categories
 - Optimizing Memory Usage

- **Pandas IO Tools**

- Reading CSV Files
- Reading Excel Files
- Reading JSON Data
- Writing Data to CSV and Excel

- **Data Cleaning and Transformation**

- Handling Missing Values
- Removing Duplicates
- Data Type Conversion
- Data Transformation Techniques

Data visualization with Python Library

- **Introduction to Matplotlib**

- What is Matplotlib?
- Importance of Data Visualization
- Installing and Importing Matplotlib
- Understanding pyplot Module
- Creating Your First Plot

- **Basic Plotting Concepts**

- Line Plots
- Plot Titles and Labels
- Adding Legends
- Customizing Axes
- Grid and Styling Options

- **Bar Charts**

- Creating Vertical Bar Charts
- Creating Horizontal Bar Charts
- Grouped Bar Charts
- Customizing Colors and Labels
- Real-World Business Data Visualization

- **Pie Charts**

- Creating Pie Charts
- Displaying Percentages
- Exploding Pie Slices

- Customizing Labels and Legends
- Visualizing Market Share Data
- **Histogram**
 - Understanding Data Distribution
 - Creating Histograms
 - Bins and Frequency Analysis
 - Customizing Histogram Appearance
 - Analyzing Statistical Data
- **Box Plot**
 - Introduction to Box Plots
 - Understanding Quartiles and Median
 - Identifying Outliers
 - Comparing Multiple Data Sets
 - Statistical Data Analysis
- **Heat Maps**
 - Introduction to Heat Maps
 - Visualizing Correlation Data
 - Color Mapping Techniques
 - Displaying Matrix-Based Data
 - Data Pattern Analysis
- **Advanced Visualization Techniques**
 - Multiple Plots in a Figure
 - Subplots and Layout Management
 - Figure Size Customization
 - Saving Charts as Images
 - Exporting Visualizations
- **Working with Real-World Data**
 - Visualizing CSV Data
 - Integrating Pandas with Matplotlib
 - Data Cleaning for Visualization
 - Creating Interactive Reports

Advanced Python Multithreading

- **Introduction to Multithreading**
 - What is Multithreading?
 - Need for Concurrent Programming
 - Advantages and Limitations of Multithreading
 - Real-World Applications of Multithreading
- **Multithreading vs Multiprocessing**
 - Understanding Threads and Processes
 - Differences Between Multithreading and Multiprocessing
 - Memory Sharing Concepts
 - Performance Comparison
 - When to Use Threads vs Processes
- **Thread Class in Python**
 - Introduction to threading Module
 - Creating Threads Using Thread Class
 - Starting and Managing Threads
 - Naming Threads

Advanced Python Webscraping

- **Introduction to Web Scraping**
 - What is Web Scraping?
 - Applications of Web Scraping
 - Data Collection from Websites
- **Understanding Website Structure**
 - Introduction to HTML
 - HTML Tags and Elements
 - Inspecting Web Pages Using Browser Developer Tools
 - Understanding DOM Structure
- **Python Requests Module**
 - Introduction to Requests Library
 - Sending HTTP Requests
 - GET and POST Methods
 - Handling Response Objects

- Response Content and Headers
- **HTTP Status Codes**
 - Understanding HTTP Protocol
 - Success Responses (200 Series)
 - Redirection Responses (300 Series)
 - Client Errors (400 Series)
 - Server Errors (500 Series)
 - Error Handling Techniques
- **Beautiful Soup Library**
 - What is Beautiful Soup?
 - Installing Beautiful Soup
 - Creating BeautifulSoup Objects
 - Parsing HTML Documents
 - Navigating HTML Elements
- **HTML Parsing Techniques**
 - HTML Parser Overview
 - Using html.parser
 - Using lxml Parser
 - Parser Comparison and Performance
- **Extracting Data from Web Pages**
 - Using get_text() Method
 - Extracting Text Content
 - Accessing HTML Attributes
 - Extracting Links and Images
- **Searching HTML Elements**
 - find() Method
 - find_all() Method
 - Searching by Tag Name
 - Searching by Class and ID
- **Data Cleaning and Transformation**
 - Removing Unwanted Content
 - Cleaning Raw Text Data
 - Formatting Extracted Information
 - Converting Raw Data into Structured Data

- **Saving Scraped Data**

- Saving Data to Text Files
- Exporting Data to CSV Files
- Working with Excel Files

- **Web Scraping Project**

- Scraping Data from E-Commerce Websites
- Extracting Product Names and Prices
- Collecting Ratings and Reviews
- Creating Structured Datasets
- Generating Reports from Scraped Data

Real time Webscraping project

Web Scraping from E-Commerce Websites

Python Projects

- **Ebook Audio Reader**

- Introduction to Text-to-Speech (GTS)
- Reading PDF and Text Files
- Converting Text into Audio
- Voice Customization and Controls
- Building a User-Friendly Interface

- **Todo List Application**

- Create, Update, and Delete Tasks
- Task Status Management
- Priority-Based Task Organization
- Data Storage Using Files or Database
- Building a Simple GUI Application

- **Railway Ticket Management System**

- User Registration and Login
- Train Search and Seat Availability
- Ticket Booking and Cancellation
- Passenger Information Management

- **Bank Management System**

- Customer Account Creation
- Deposit and Withdrawal Operations
- Balance Inquiry and Account Statements
- Fund Transfer Functionality
- Transaction History Management

- **Super Market Bill Generation System**

- Product Inventory Management
- Adding and Removing Products
- Automatic Bill Generation
- Tax and Discount Calculation
- Invoice Printing and Export

Git&GitHub

• Introduction to GitHub

- What is GitHub?
- Importance of GitHub in Software Development
- Features and Benefits of GitHub
- GitHub Use Cases for Developers and Teams

• GitHub Account Creation and Setup

- Creating a GitHub Account
- Profile Configuration
- Security Settings and Two-Factor Authentication
- GitHub Dashboard Overview

• Understanding GitHub Repositories

- What is a Repository?
- Public vs Private Repositories
- Creating New Repositories
- Repository Structure and Navigation

• Working with GitHub Projects

- Uploading Existing Projects
- Pushing Local Projects to GitHub
- Cloning and Pulling Projects from GitHub
- Managing Repository Files

• README File Creation

- Importance of README Files
- Markdown Syntax Basics
- Project Description and Documentation
- Creating Professional README Files

• Introduction to Git

- What is Git?
- Version Control Concepts
- Benefits of Git in Development
- Git Workflow Overview

• Git Installation and Configuration

- Installing Git on Windows, Linux, and macOS
- Verifying Installation
- Git Configuration Commands
- Setting Username and Email

- **Essential Git Commands**

- git init – Initializing a Repository
- git clone – Cloning Remote Repositories
- git status – Checking Repository Status
- git add – Staging Files for Commit
- git commit – Saving Changes with Messages
- git log – Viewing Commit History

- **Working with Remote Repositories**

- Connecting Local Repository to GitHub
- git remote add
- git push – Uploading Changes
- git pull – Fetching and Merging Updates

- **Branching and Collaboration**

- Understanding Branches
- Creating and Switching Branches
- Merging Branches
- Introduction to Pull Requests

- **Git vs GitHub**

- Differences Between Git and GitHub
- Version Control vs Code Hosting
- When to Use Git and GitHub
- Real-World Development Workflow

Django for Python Developer

• Introduction to Django

- What is Django?
- Features and Advantages of Django
- MVT (Model-View-Template) Architecture
- Django Use Cases and Real-World Applications

• Python Package Management (PIP)

- Introduction to PIP
- Installing Python Packages
- Managing Dependencies
- Requirements.txt File

• Django Installation and Environment Setup

- Installing Django Framework
- Verifying Django Installation
- Creating and Using Virtual Environments
- Django Project Structure Overview

• Creating Django Projects and Applications

- Creating a Django Project
- Understanding Project Files
- Creating Django Applications
- Registering Applications
- Working with Multiple Applications

• Django Management Commands

- django-admin Commands
- manage.py Commands
- runserver
- startproject
- startapp
- makemigrations and migrate
- createsuperuser

• Django Configuration Files

- Understanding settings.py
- Installed Apps Configuration
- Database Settings

- Static and Media Files Configuration
- **Django URL Routing**
 - Understanding urls.py
 - URL Patterns and Path Routing
 - Application-Level URLs
 - Including URL Configurations
 - Dynamic URL Parameters
- **Django Views**
 - Function-Based Views
 - Handling HTTP Requests and Responses
 - Rendering Templates
 - Passing Data to Templates
 - Creating Multiple Views
- **Django Templates**
 - Template Configuration
 - Creating HTML Templates
 - Template Variables
 - Template Tags and Filters
 - Template Inheritance
 - Including Static Files
 - Displaying Images and Media Files
- **Django Models and Database Integration**
 - Introduction to Models
 - Creating Database Models
 - Field Types and Model Relationships
 - Using MySQL Database with Django
 - CRUD Operations using Models
- **Django Migrations**
 - Understanding Database Migrations
 - makemigrations Command
 - migrate Command
 - Updating Database Schema
 - Migration Best Practices
- **Django Admin Panel**

- Creating Superuser
- Registering Models in Admin
- Customizing Admin Interface
- Managing Database Records

- **Project: First Django Web Application**

- Create and Configure a Django Project
- Run the Development Server
- Create Applications and Views
- Configure URLs
- Display Dynamic Content
- Manage Multiple Applications

Django Projects

- Student Management System (Django) – A web application built with Django to manage students and academic records. It allows adding, updating, deleting, and viewing student details through an admin panel and user interface.
- Portfolio Website (Django) – A personal website created using Django to showcase your profile, skills, projects, and contact details. It helps professionals present their work and achievements in an interactive, visually appealing format.

Generative AI for Developers

1. Artificial Intelligence Fundamentals

- Fundamentals of Artificial Intelligence
- Importance of AI in Modern Technology
- Real-World AI Applications
- AI, Machine Learning, and Generative AI Comparison

2. Generative AI Foundations

- Overview of Generative AI
- Business and Development Use Cases
- Popular Generative AI Platforms

3. Getting Started with ChatGPT

- ChatGPT Account Setup
- ChatGPT Workspace and Interface
- Managing Chats and Conversation History
- Free vs Premium Features

4. Getting Started with Claude AI

- Claude AI Overview
- Claude Account Setup
- Claude Workspace and Navigation
- Managing Chats and Projects

5. Prompt Engineering Essentials

- Fundamentals of Prompt Engineering
- Designing Effective Prompts
- Common Prompting Strategies
- Prompt Optimization Best Practices

6. ChatGPT for Python Programming

- Learning Python Concepts with AI
- AI-Assisted Code Generation
- Code Review and Explanation
- Practice Questions and Learning Support

7. Introduction to LLM APIs

- Large Language Model (LLM) APIs Overview
- API Authentication and Key Management
- Integrating OpenAI APIs with Python
- Prompt Submission and Response Handling
- Building AI-Powered Applications
- Developing a Basic AI Chatbot

8. AI for Career Growth

- AI-Assisted Resume Development
- Cover Letter Generation
- LinkedIn Profile Enhancement
- Interview Preparation with AI
- Professional Communication and Email Writing

9. AI for Project Documentation

- README File Generation
- Project Report Preparation
- Automated Code Documentation
- AI-Assisted Presentation Content Creation
- Documentation Standards and Best Practices

10. Exploring Modern AI Tools

- ChatGPT
- Claude AI
- Gemini AI
- Perplexity AI
- Grok AI
- NotebookLM
- Grammarly AI

HyperText Markup Language (HTML) {Recordings}

- HTML Introduction
- HTML Editor Installation
- HTML Basics
- HTML Elements
- HTML Attributes
- HTML Headings
- HTML Paragraphs
- HTML Formatting
- HTML Comments
- HTML Colors
- HTML Links
- HTML Images
- HTML Head
- HTML Forms
- HTML Tables
- HTML Lists

Cascading Style Sheets (CSS) {Recordings}

- CSS Introduction
- CSS Syntax
- CSS Selectors
- CSS Comments
- CSS Colors
- CSS Backgrounds
- CSS Borders
- CSS Padding
- CSS Text
- CSS Fonts
- CSS Icons
- CSS Links
- CSS Lists
- CSS Tables

JavaScript {Recordings}

Introduction to JavaScript

- Overview of JavaScript as a Programming Language
- Client-Side vs Server-Side JavaScript
- Key Features and Uses of JavaScript

History and ECMAScript Standards

- Evolution of JavaScript
- Understanding ECMAScript Versions
- Compatibility with Modern Browsers

Setting Up Development Environment

- Installing Node.js and npm
- Setting Up Code Editors (VS Code)
- Using Browser Developer Tools

Variables (var, let, const)

- Difference Between var, let, and const
- Scope and Hoisting Behavior
- Choosing the Right Variable Type

Data Types (Primitive and Non-Primitive)

- Primitive Types: Number, String, Boolean, Undefined, Null, Symbol
- Non-Primitive Types: Objects, Arrays, Functions
- Type Checking using typeof

Operators (Arithmetic, Comparison, Logical, Assignment)

- Arithmetic: +, -, *, /, %, **
- Comparison: ==, ===, !=, !==, >, <, >=, <=
- Logical: &&, ||, !
- Assignment: =, +=, -=, *=, /=

Type Conversion and Coercion

- Explicit Conversion: Number(), String(), Boolean()
- Implicit Coercion in Expressions
- Common Pitfalls and Best Practices

Control Statements (if, else, switch)

- Conditional Execution Using if and else
- Switch Statement for Multiple Cases
- Nesting and Combining Conditions

Loops (for, while, do...while, for...in, for...of)

- Iterating with for and while Loops
- do...while Loop Usage
- Iterating Objects and Arrays with for...in and for...of

Functions (Declarations, Expressions, Arrow Functions)

- Function Declarations vs Function Expressions
- Arrow Functions and Their Syntax
- Returning Values and Scope Considerations

Scope (Global, Local, Block)

- Understanding Variable Scope
- Global vs Local Variables
- Block Scope with let and const

Hoisting

- How var, let, const, and Functions are Hoisted
- Hoisting Behavior Differences
- Best Practices to Avoid Hoisting Issues

Closures

- Functions Retaining Access to Outer Scope
- Practical Use Cases: Data Privacy, Callbacks
- Memory Considerations

Objects and Object Methods

- Creating Objects Using Literals and Constructors
- Accessing and Modifying Object Properties
- Built-in Object Methods and Custom Methods

Arrays and Array Methods

- Creating and Accessing Arrays
- Common Methods: push, pop, shift, unshift, splice, slice
- Iteration Methods: forEach, map, filter, reduce

Strings and String Methods

- String Creation and Manipulation
- Methods: length, charAt, slice, substring, replace, split, trim
- Template Literals and String Interpolation

Template Literals

- Using Backticks for Multi-line Strings
- Embedding Expressions with \${}
- Creating Dynamic Strings

Destructuring (Array and Object)

- Extracting Values from Arrays and Objects
- Assigning to Variables in One Statement
- Nested Destructuring

Spread and Rest Operators

- Spread (...) to Expand Arrays and Objects
- Rest (...) to Collect Function Arguments
- Combining Spread and Rest with Functions and Arrays

Error Handling (try, catch, finally, throw)

- Using try and catch to Handle Exceptions
- Finally Block Execution
- Throwing Custom Errors

DOM Manipulation

- Selecting Elements with querySelector and getElementById
- Changing Element Content and Attributes
- Adding and Removing Elements Dynamically

Events and Event Handling

- Understanding Event Types (click, input, load, etc.)
- Adding Event Listeners
- Event Bubbling and Delegation

Projects

1. Dictionary Management System
2. Retail Billing and Invoice Management System
3. ATM Transaction Management System
4. Banking Operations Management System
5. AI-Powered Text-to-Speech eBook Reader
6. Exploratory Data Analysis and Visualization Project
7. Real-Time E-Commerce Data Extraction Platform
8. Task Management Web Application with Django
9. Professional Portfolio Website with Django
10. Student Information Management System with Django

What We Provide

- Lifetime Access to Recorded Sessions
- Daily Doubt-Clearing Sessions
- Comprehensive Topic-Wise Study Materials
- 30+ Daily Job Opportunities and Updates
- Mock Interview Preparation Sessions
- Professional Resume Building Assistance
- 2 Hours of Live Online Training Every Day
- Hands-On Practical Learning Approach
- 1000+ Python, SQL, Django, and Interview Questions
- Daily Assignments and Practice Tasks
- LinkedIn Profile Optimization and Activity Tracking
- Course Duration: 2 Months
- 10 Industry-Oriented Real-Time Projects
- Course Completion Certificate
- GitHub Portfolio Development
- Career Roadmap Sessions
- Continuous Learning and Community Support

Job Roles You Can Apply For :

- Python Developer ☆ ☆ ☆ ☆ ☆ (Primary role)
- Junior Python Developer
- Backend Python Developer (Junior) (*especially with Django*)
- Django Developer
- Web Scraping Developer
- AI-Assisted Python Developer

Data-Related Roles

- Data Analyst Intern
- Junior Data Analyst
- Python Data Analyst (Entry Level)

AI-Related Roles

- AI Application Developer (Junior)
- Prompt Engineer (Entry Level)
- Generative AI Associate

Freelancing Opportunities

Students can also work as freelancers by offering:

- Python scripting
- Automation
- Web scraping
- Django applications
- AI-powered business tools

Companies Hiring Python Developers & AI Developers

- Tata Consultancy Services (TCS)
- Infosys
- Wipro
- HCLTech
- Tech Mahindra
- Accenture
- Capgemini
- Cognizant
- LTIMindtree
- Mphasis

What Recruiters Expect Beyond This Course To maximize your chances of getting hired:

- Upload all your projects to GitHub with professional README files.
- Build a strong portfolio with Python, Django, AI, and automation projects.
- Practice Python coding problems and Object-Oriented Programming concepts daily.
- Strengthen your knowledge of NumPy, Pandas, and Data Analysis.
- Learn to integrate AI APIs into Python applications.
- Prepare Python, Django, SQL (basic), Git, and AI interview questions.
- Keep your LinkedIn profile active and showcase your projects regularly.
- Participate in coding challenges and contribute to open-source projects.
- Develop strong problem-solving and debugging skills.
- Continue learning modern AI tools and Python libraries as technology evolves.

Career Opportunities After This Course

After successfully completing this program, you can apply for roles such as:

- Python Developer
- Junior Python Developer
- Django Developer
- Backend Python Developer
- Web Scraping Developer
- AI Application Developer
- Generative AI Developer (Entry Level)
- Python Data Analyst (Entry Level)
- Software Engineer (Python)